

SMART EXAM GRADE PREDICTOR

Java-Based GUI Application

A console-free desktop tool for predicting grades, calculating target marks, and validating exam inputs.

Submitted By

Awatans Asthana Course: Java Programming | Branch: ECE | VIT
Bhopal University

Academic Year: 2026

JAVA
SWING

Predict

Validate

Analyze

Report

LEARN | INNOVATE | GROW

Project Overview

What the application does and why it is useful

THE PROBLEM

Students often want to know both their likely final grade and the marks needed for a target grade.

- Manual calculations are repetitive and error-prone.
- Different mark components need to be combined consistently.
- Target-grade planning is difficult without a calculator.

THE SOLUTION

1. Enter marks
2. Validate inputs
3. Calculate total
- 4. Predict grade**
5. Find target marks

Project-defined rule set: 40 marks Internal + 30 marks Mid-Term + 30 marks End-Semester.

Objectives

Key outcomes of the Java application

01 Predict Grade

Combine entered/expected marks and return a predicted total and grade.

02 Target Marks

Calculate the end-semester marks required for a selected target grade.

03 Validate Input

Reject blank, non-numeric, negative, or out-of-range mark entries.

04 Use OOP

Separate data, calculation, validation, and reporting responsibilities.

05 Simple GUI

Provide a beginner-friendly desktop interface using Java Swing.

06 Extensible Design

Keep logic in reusable Java classes for future upgrades.

Functional & Non-Functional Requirements

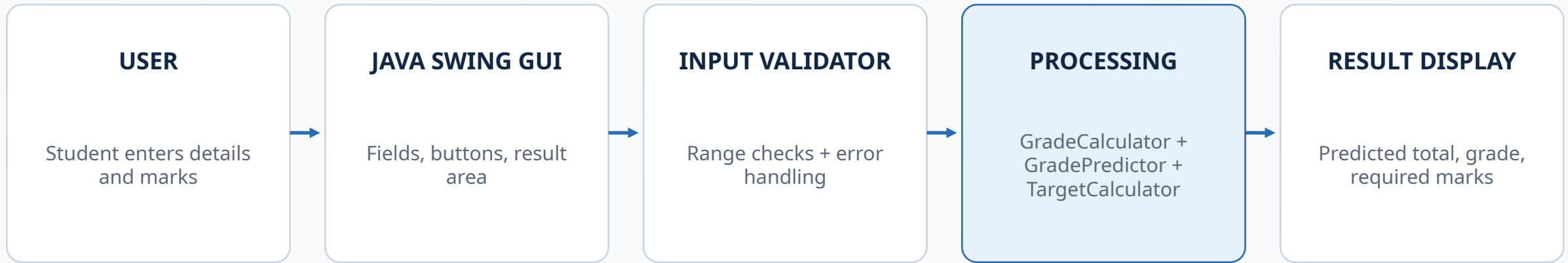
Mapped to the VITyarthi project expectations

FUNCTIONAL	NON-FUNCTIONAL
Grade Prediction Uses current + expected marks to predict a grade.	Usability Clear labels, buttons, and simple navigation.
Target Grade Calculation Finds required end-semester marks for a chosen grade.	Reliability Range checks and exception handling avoid invalid calculations.
Input Validation Checks ranges and handles invalid entries with error messages.	Maintainability Logic split across focused Java classes.
Result Display Shows total marks and predicted grade in the GUI.	Resource Efficiency Lightweight desktop GUI; no database/server required.

The VITyarthi document requires at least 3 functional modules, 4+ non-functional requirements, and 5-10 meaningful code modules/classes.

System Architecture

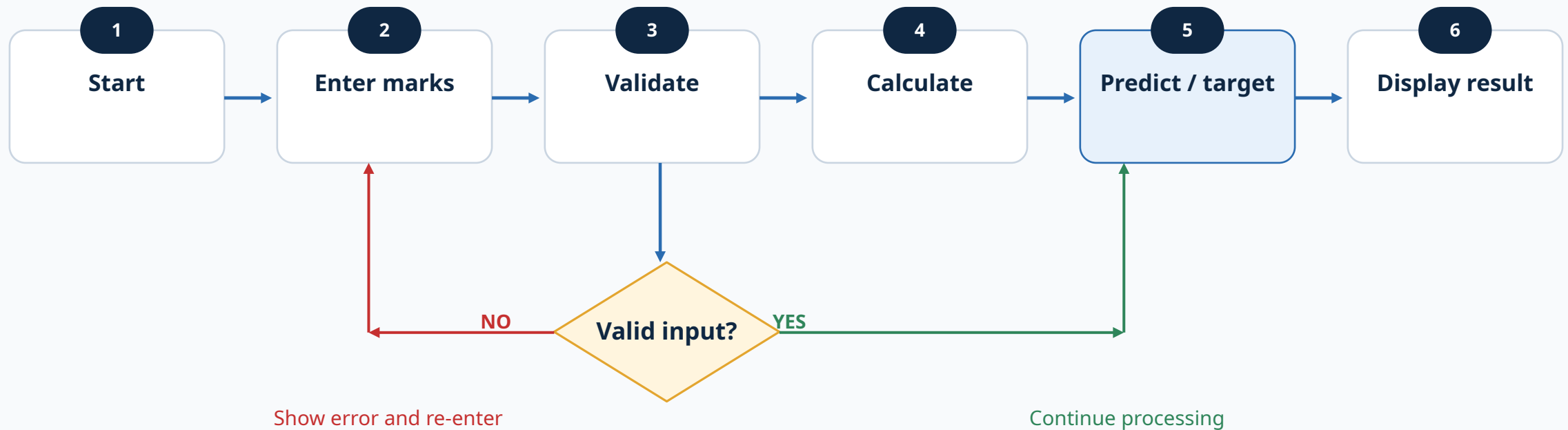
High-level flow of the Java Swing application



Supporting classes in the source structure include Student, Subject, PerformanceAnalyzer, InputValidator, FileManager, GradeCalculator, GradePredictor, and TargetCalculator.

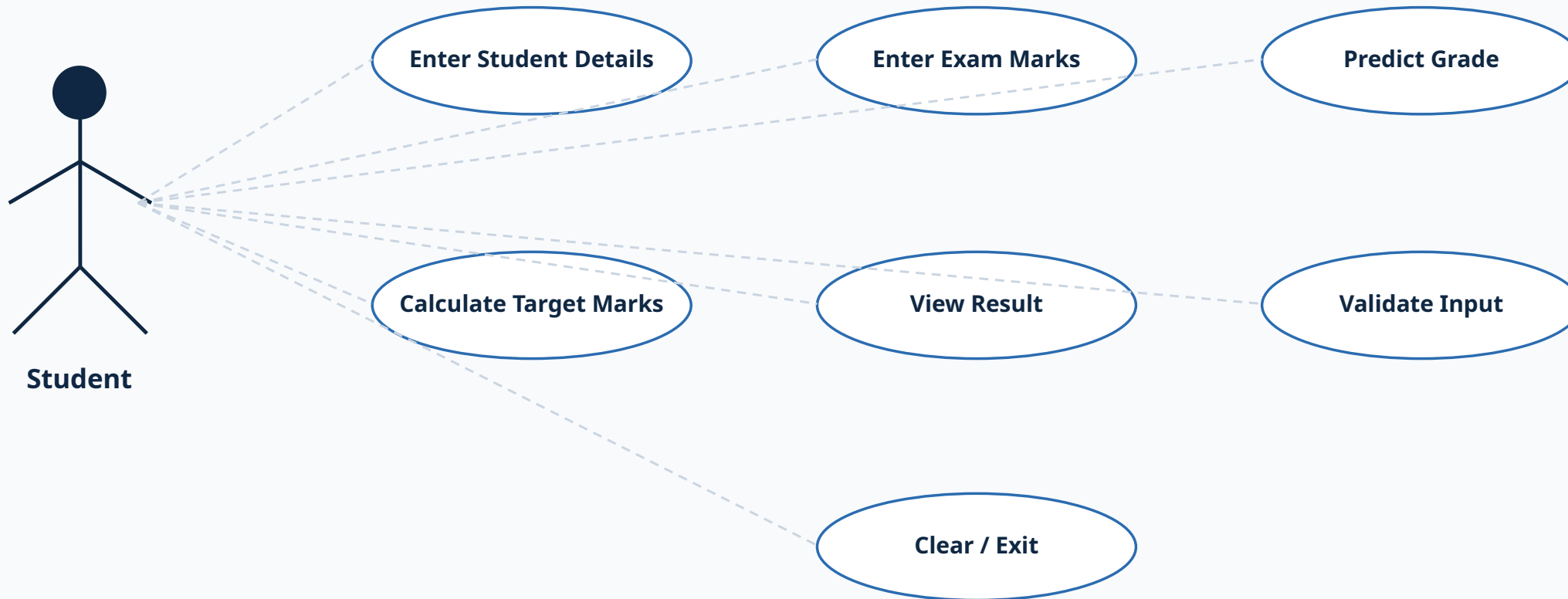
Workflow Diagram

Input -> validation -> calculation -> prediction -> result



Use Case Diagram

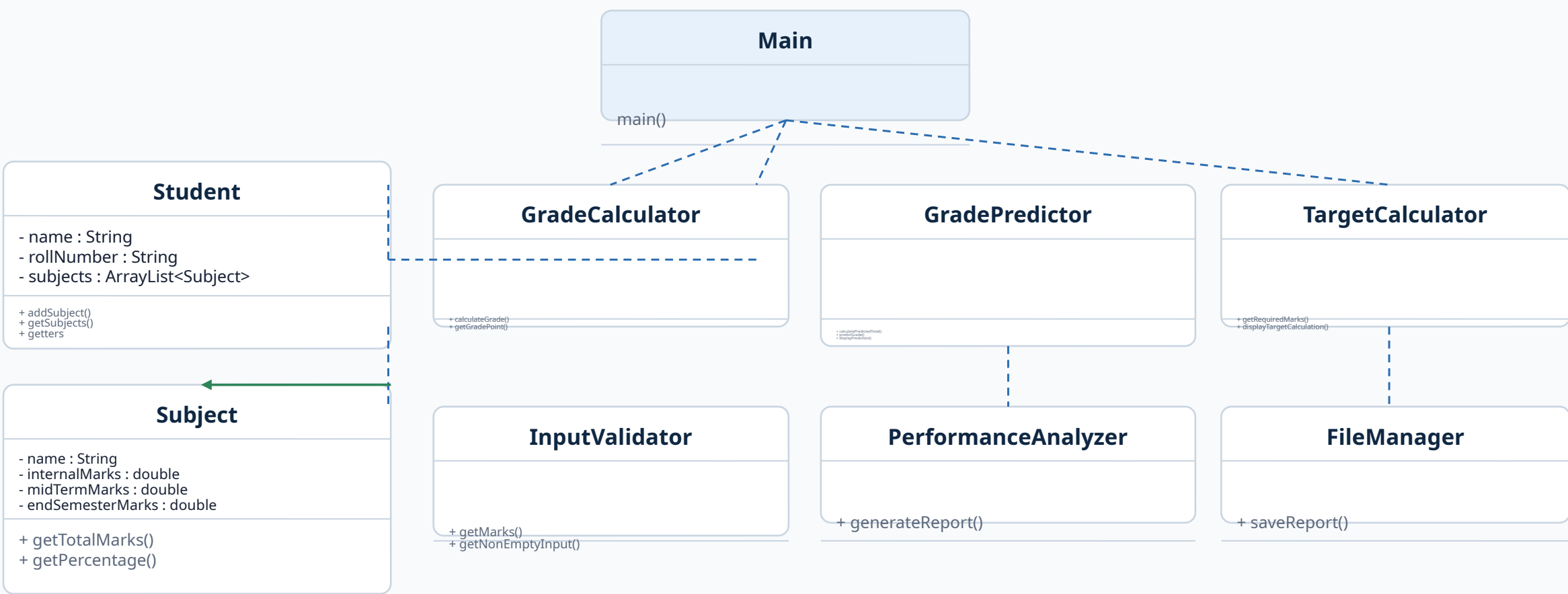
Primary interactions between the student and the application



Use-case notation is simplified for presentation clarity; the interactions reflect the functions available in the current project code.

Class Diagram

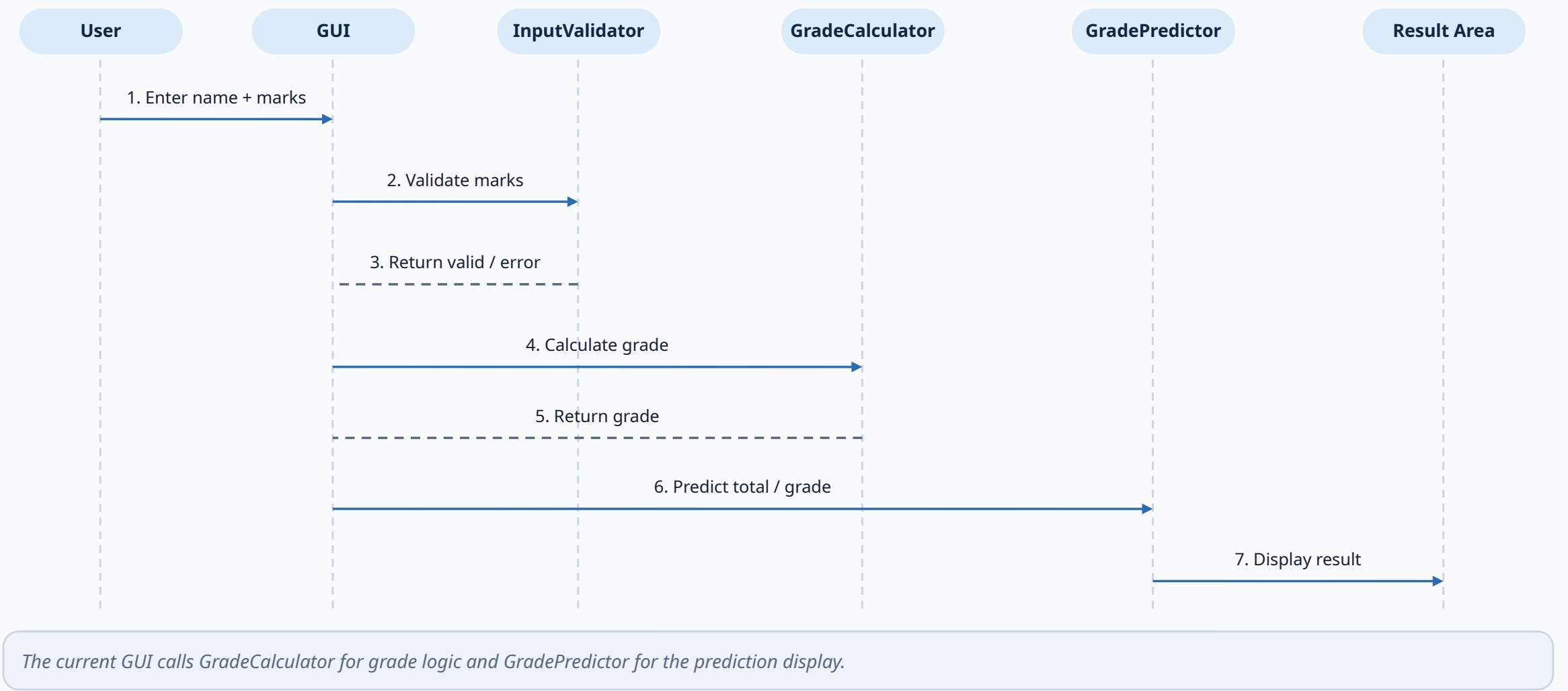
Core Java classes and their responsibilities



Student contains a collection of Subject objects; service/calculation classes provide focused operations.

Sequence Diagram

Typical “Predict Grade” interaction



Grading Rules & Calculation Logic

Project-defined grading bands and mark distribution



Important: these grading bands and the 40/30/30 distribution are rules defined for this project, not stated in the VITyarthi guideline PDF.

Application Interface & Sample Result

Representative view of the implemented Swing interface

SMART EXAM GRADE PREDICTORJava Swing GUI

Student Name	Roll Number
Awatans	101
Subject	Internal Marks
Java	35
Mid-Term Marks	Expected End-Sem
25	25

Predict Grade

Target Grade

Clear

Exit

SAMPLE PREDICTION

Predicted Total

85 / 100

Predicted Grade

A

Example: 35 + 25 + 25 = 85 -> A

TARGET GRADE EXAMPLE

Current marks: 35 + 25 = 60

Target: A -> Required End-Sem: 20 / 30

Implementation & Project Structure

Modular Java source structure

Main.java

GUI + event handling

Subject.java

Subject data model

Student.java

Student data + subject list

GradeCalculator.java

Grade rules + grade points

GradePredictor.java

Predicted total and grade

TargetCalculator.java

Required marks logic

PerformanceAnalyzer.java

Multi-subject report logic

InputValidator.java

Input and range checks

FileManager.java

Text-file report saving

Technology stack: Java, Java Swing, OOP, ArrayList, exception handling, and text-file handling.

Testing & Validation

Functional test cases for the implemented calculator

Test Case	Input	Expected Result	Status
Valid marks	35, 25, 25	85 / 100, Grade A	Pass
Boundary marks	40, 30, 30	100 / 100, Grade A+	Pass
Negative input	-5	Error message	Pass
Over-range input	45 (Internal)	Error message	Pass
Non-numeric input	abc	Validation error	Pass
Target grade	35, 25, target A	20 / 30 required	Pass

Validation is performed in the GUI before calculations. Numeric parsing is protected with exception handling.

Design Decisions, Storage & Trade-offs

Why the project is implemented this way

Java Swing

Chosen for a desktop GUI without requiring a web server or external framework.

OOP Classes

Each responsibility is placed in a separate class for readability and reuse.

ArrayList

Provides a simple in-memory collection for multiple subjects.

File Storage

FileManager supports plain-text report output; no relational database is used.

Validation First

Inputs are checked before calculation to reduce invalid results.

Extensible Logic

Grade rules are isolated in calculation classes for easy future change.

Because storage is file-based rather than relational, an ER diagram is not applicable to the current implementation.

Challenges & Key Learnings

What the project demonstrates from a Java perspective

CHALLENGES

- Connecting multiple Java classes without losing simplicity.
- Handling numeric parsing and out-of-range marks safely.
- Designing a GUI that is clear for first-time users.
- Keeping calculation logic separate from interface code.

LEARNINGS

- Applying OOP to a real-world academic problem.
- Using Java Swing components and event listeners.
- Writing reusable methods for prediction and target calculations.
- Using Git and GitHub for version control and submission.

The VITyarthi rubric explicitly values implementation quality, design/documentation, version control, and learning outcomes.

Future Enhancements

Ideas for the next version

01 CGPA Predictor

Add credits and grade-point calculations across subjects.

02 Charts & Analytics

Show subject-wise performance using charts.

03 PDF Export

Generate a polished downloadable report.

04 Database Support

Store student history using SQLite/MySQL.

05 Login System

Provide separate student and admin access.

06 Course Profiles

Allow different grading rules for different courses.

Submission Readiness & GitHub

How the project maps to the VITyarthi submission requirements

- ✓ **3+ Functional Modules** Grade prediction, target calculation, input validation
- ✓ **4+ Non-Functional Requirements** Usability, reliability, maintainability, resource efficiency
- ✓ **5-10 Meaningful Classes** 9 Java source files in src/
- ✓ **Design Artifacts** Architecture, workflow, use case, class, sequence
- ✓ **GitHub Repository** README.md + statement.md + src/ source files

GitHub: github.com/awatansasthana-learn/SmartExamGradePredictor

References

Sources used for project requirements and implementation framing

1. VITyarthi - Build Your Own Project: General Project Instructions & Submission Guidelines

Used for project requirements, design artefacts, GitHub structure, report sections, and evaluation rubric.

2. Java Platform, Standard Edition - Swing / Core Java concepts

Used as the conceptual basis for the desktop GUI, OOP structure, collections, validation, and file handling.

Project grading bands and mark distribution shown in this deck are explicitly defined for this project.

THANK YOU